

HERMES AGENT COLLECTION

Hermes Agent: interfaces, administration, and technical operation

Technical guide to Desktop, Dashboard, CLI, TUI, and Messaging Gateway

\$ hermes --tui

Desktop | Dashboard | CLI | TUI | Gateway

same core - different surfaces

Hermes Agent v0.20.0 - release 2026.8.3

Café com Dados & Gatos

Contents

- Introduction
- 1. Hermes Agent conceptual architecture
- 2. Five layers for understanding the agent
- 3. Hermes Desktop
- 4. Security, memory, and context in Desktop
- 5. Voice, execution, and Desktop administration
- 6. Web Dashboard
- 7. Capabilities, extensions, and integrations in Dashboard
- 8. CLI and TUI
- 9. Messaging Gateway and remote connections
- 10. Choosing the interface
- 11. Security and best practices
- 12. Glossary
- Official references

Introduction

Hermes Agent offers several surfaces for accessing the same agent core. The Desktop application, Web Dashboard, CLI, and TUI are not different agents. When they use the same Profile, these interfaces share configuration, models, credentials, memory, skills, and sessions. The Messaging Gateway also uses this core, but operates mainly as a continuous service for external channels.

This e-book consolidates technical aspects of Hermes Agent interfaces and configuration. It is intended as a direct-reference guide for students and professionals, focusing on concepts, components, operation, and security.

The content distinguishes three types of information:

- ****Documented behavior:**** described in the official Hermes Agent documentation.
- ****Interface observation:**** verified in Hermes Agent v0.20.0, release 2026.8.3.

- **Technical interpretation:** explanations used to relate components and make them easier to understand.

Interfaces, menu names, and options may change between versions. Before applying a configuration in production, compare your local installation with the official documentation.

1. Hermes Agent conceptual architecture

1.1 One core, multiple surfaces

Hermes Agent separates the agent mechanism from the interfaces used to operate it. This architecture allows you to start a session in Desktop, resume it in the terminal, and inspect it in Dashboard, provided all surfaces point to the same Profile and storage.

Main surfaces:

- **Hermes Desktop:** native application focused on chat, files, previews, and task execution.
- **Web Dashboard:** browser-based administrative panel for sessions, configuration, observability, and integrations.
- **CLI:** command-line interface for reproducible operations, diagnostics, and automation.
- **TUI:** interactive text-mode interface that runs inside the terminal.
- **Messaging Gateway:** service that keeps the agent connected to channels such as Telegram, Discord, Slack, WhatsApp, and email.

1.2 Shared components

When the active Profile is the same, the surfaces may share:

- `config.yaml` and other Profile settings;
- secrets and credentials stored in the Hermes environment;
- main model and auxiliary models;
- persistent memory and user profile;
- skills, toolsets, and MCP servers;
- sessions and history;
- recurring tasks;
- logs and operational state related to the Profile.

Purely visual elements remain surface-specific, such as themes, panel layout, Desktop shortcuts, and TUI display preferences.

1.3 The role of Profiles

A Profile represents an isolated logical Hermes instance. Different Profiles can maintain independent configuration, keys, memory, sessions, skills, recurring tasks, logs, and gateway state.

Changing interface does not create a new agent. Changing Profile may change the entire operational context. For this reason, identifying the active Profile is a fundamental check before changing configuration, credentials, or automations.

1.4 Quick comparison

Surface	Nature	Main use	Strength	Typical limitation
Desktop	Native application	Daily work, chat, files, artifacts	Integrated visual experience	Requires application installation
Dashboard	Administrative web app	Configuration, observability, audit	Centralized system view	Does not replace the Desktop working experience
CLI	Terminal commands	Installation, scripts, servers, diagnostics	Reproducibility and automation	Requires terminal proficiency
TUI	Interactive text interface	Conversation and operation in terminal	Lightweight, continuous navigation	Less visual than Desktop
Messaging Gateway	Continuous service	External channel connectivity	Persistent multichannel operation	Not a complete administrative interface

2. Five layers for understanding the agent

2.1 Model

The model is the AI system that interprets requests and generates responses. Hermes can use a main model and auxiliary models for specialized tasks. The provider hosts the model and defines access conditions, billing, and limits; provider and model are therefore not synonyms.

2.2 Tools

Tools allow the agent to perform actions. Examples include terminal access, file reading and writing, browser use, search, APIs, media generation, and MCP integrations. A model without tools only generates text; tools connect model decisions to actions in the environment.

2.3 Context

Context is the information sent to the model in a turn. It may include system instructions, project rules, persistent memory, session messages, tool results, and uploaded files. All of this competes for the available context window.

2.4 Memory

Persistent memory stores selected facts for reuse in future sessions. It is not the complete conversation history. Its goal is to preserve stable, useful information rather than every message.

2.5 Permissions

Permissions define how far the agent may act automatically and when it must request approval. They reduce the risk of destructive commands, improper access to internal resources, and exposure of sensitive information.

3. Hermes Desktop

3.1 Purpose and architecture

Hermes Desktop is a native application for Windows, macOS, and Linux. It uses the same core, sessions, and configuration available in terminal interfaces. The application prioritizes conversation, visual tracking of tools, file navigation, artifacts, and task execution.

Desktop starts its own local backend and can also connect to a remote backend. This backend must not be confused with the model provider or the Messaging Gateway used by external channels.

3.2 Main model, auxiliary models, and Mixture of Agents

The Model area defines the provider and default model for new sessions. When the provider supports reasoning control, the configured level may change latency, cost, and performance on complex tasks.

Auxiliary models can specialize internal functions:

- Vision: image interpretation;
- Web extract: page extraction and summarization;
- Compression: context summarization in long conversations;
- Skills hub: skill search;
- Approval: command evaluation for intelligent approval;
- MCP: routing tools exposed by MCP servers;
- Title generation: automatic session-title generation;
- Curator: skill review and curation.

When an option indicates automatic use or use of the main model, the auxiliary function inherits the main model. Specialization may reduce cost or improve specific tasks, but it also increases configuration complexity.

Mixture of Agents coordinates multiple agents or models and uses an aggregator to combine or evaluate responses. This pattern is useful for problems that benefit from different perspectives, but can be excessive for simple tasks.

3.3 Conversation and attachments

The Chat area controls the conversation experience, including:

- preview limit and local image loading;
- default personality for new sessions;
- time zone for dates, schedules, and relative references;
- display of reasoning blocks exposed by the backend;
- handling of image attachments.

The image-loading limit is measured in megabytes and does not represent the model's context window. Displaying reasoning blocks also does not mean unrestricted access to the model's entire internal process.

3.4 Appearance and technical density

The Appearance area includes language, light/dark mode, themes, scale, terminal font, transparency, and background. These options change readability and ergonomics, but do not change the agent's intelligence.

Tool-call display commonly offers a friendly view and a technical view. The technical view exposes more detail about inputs, outputs, and steps, making it useful for auditing and diagnostics.

3.5 Workspace and work scope

The Workspace defines the agent's working territory:

- default working directory;
- automatic Git repository discovery;
- allowed discovery roots;
- paths excluded from scanning;
- project execution mode;
- persistent shell;
- controlled forwarding of environment variables;
- file-read limits.

A persistent shell preserves the current directory and state between commands when supported by the backend. Forwarding environment variables requires care because it can expose secrets to tools. Read limits prevent very large files from consuming an excessive portion of context.

4. Security, memory, and context in Desktop

4.1 Approvals and safety barriers

The Safety area defines how Hermes authorizes actions:

- Approval Mode;
- Approval Timeout;
- Confirm MCP Reloads;
- Command Allowlist;
- Redact Secrets;
- controls for access to private URLs and local addresses;
- File Checkpoints before file changes.

Checkpoints help recover changes but do not replace version control or backups. Allowing internal addresses expands agent capability and also increases the risk surface.

4.2 Persistent memory, user profile, and history

Three concepts should remain separate:

- ****Active context:**** material sent to the model in the current turn.
- ****Persistent memory:**** short, stable facts reused in future sessions.
- ****Session history:**** the complete persisted record of messages and events.

The native provider can store memory notes in MEMORY.md. Stable user preferences may be stored separately in USER.md. This separation distinguishes project/environment knowledge from interaction preferences.

Memory and profile limits are shown in characters. Values observed in the documented version, such as 2,200 characters for memory and 1,375 for profile, are examples from that installation rather than universal recommendations. Compact memory reduces permanent context consumption.

4.3 Context engine and compression

The context engine manages long conversations. The compressor replaces eligible older material with a summary when usage approaches the configured limit.

Compression does not increase the model's maximum context window. It exchanges older details for a condensed representation. This preserves continuity but may lose precision.

Relevant parameters include:

- Auto-Compression;
- Compression Threshold;
- Compression Target;
- Protected Recent Messages.

With threshold = 0.5 and target_ratio = 0.2, the nominal budget for the recent tail is approximately 10% of the effective window (0.5 x 0.2). Protected messages can make the final result exceed that target.

In session compression mode, condensed material may remain archived in a recoverable form. Even so, a summary should not be treated as perfect recovery of the original text.

4.4 Common conceptual mistakes

- Persistent memory is not the complete history.
- Context contains more than just the user's typed message.
- Compression does not guarantee preservation of every detail.
- A larger window may increase cost and latency.
- The memory budget is not the same as the context-window size.

5. Voice, execution, and Desktop administration

5.1 Voice and transcription

The Voice area controls:

- text-to-speech provider;
- transcription enablement;
- display of raw recognized text;
- speech-to-text provider;
- automatic reading of responses;
- voice used for local synthesis;
- transcription model and language;

- voice shortcut;
- maximum recording duration.

Larger local models usually require more resources and may provide better accuracy. The configured language affects speech recognition.

5.2 Execution, limits, and backends

The Advanced area contains execution controls such as:

- keeping the machine awake during long tasks;
- quick input through a global shortcut;
- enabled toolsets;
- local or isolated execution backend;
- command timeout;
- base images for isolated environments;
- terminal-output and file-read limits;
- number of checkpoints;
- maximum agent steps;
- retries after transient API failures;
- provider service tier;
- tool-use policy.

Backends such as Docker, Singularity, Modal, and Daytona can separate execution from the main system. The degree of isolation depends on configuration and should not be assumed from the backend name alone.

5.3 Subagents

Subagents allow parts of a task to be delegated. Main controls include model, provider, reasoning effort, turn limit, parallelism, and maximum execution time.

More parallelism can reduce total time for independent tasks, but it increases resource use, cost, and coordination complexity.

5.4 Notifications

Notifications can indicate pending approval, required input, completed response, failure, background-task completion, and credit alerts. They are especially useful for long-running executions.

5.5 Billing and consumption

The Billing area summarizes plan, balance, renewal, credit purchases, consumption, and monthly cap. The panel refers to the connected service/account. Providers configured with personal keys may bill directly on their own platforms.

Financial data, identifiers, and payment methods should be hidden in public screenshots.

5.6 Providers and endpoints

The Providers area centralizes account authentication, API keys, and compatible custom endpoints. The provider controls hosting, availability, billing, and limits. The model is an option offered by that provider.

5.7 Backend connection

Desktop can operate with:

- a local backend managed by the application;
- Hermes Cloud;
- a remote backend run by `hermes serve`;
- a remote connection protected by SSH or another documented mechanism.

The Desktop backend orchestrates the agent. The provider performs model inference. The Messaging Gateway maintains communication channels. These three components have different responsibilities.

Backends accessible outside the local machine require authentication, network protection, and proper configuration. A service bound to 127.0.0.1 accepts connections only from the local machine.

5.8 Keyboard shortcuts

Shortcuts speed up session creation, model selection, voice, message queue, file references, command palette, panels, and Profile switching. Conflicting key combinations should be resolved before relying on a shortcut in critical workflows.

5.9 Tools, keys, and integrations

Tool credentials are separate from model credentials. A search key enables queries; a voice key enables synthesis; a GitHub credential may authorize repository operations.

Secrets should be stored in the appropriate mechanism and never copied into public documentation, prompts, or screenshots.

5.10 Plugins, skills, tools, and MCP

These terms refer to different mechanisms:

- **Plugin:** software extension that adds components or behavior.
- **Skill:** set of instructions and specialized resources for a class of tasks.
- **Tool:** concrete capability invoked by the agent.
- **MCP:** protocol and mechanism for exposing tools and resources through integrated servers.

5.11 Archiving and removal

Archiving removes a conversation from the main list without deleting its history. Pinned conversations may remain outside automatic archiving rules.

The About area shows version, updates, and uninstall options. Removal may affect only the graphical interface, remove the agent while preserving data, or delete all data. Full removal is destructive and requires careful scope verification.

6. Web Dashboard

6.1 Purpose and access

The Web Dashboard is Hermes's administrative panel. By default, hermes dashboard starts a local service at `http://127.0.0.1:9119`. While bound to loopback, it accepts access only from the local machine.

Dashboard prioritizes administration and observability. It can read and modify the same data used by Desktop and terminal interfaces. The Chat tab embeds the real Hermes TUI through a pseudoterminal; the other pages are administrative web interfaces.

6.2 Sessions

The Sessions area lists human conversations and automated executions. It can search, filter by source, open histories, inspect tools, rename, export, archive, and administer sessions according to the installed version.

Session and context are not synonyms. The session is the persisted record; context is the active subset sent to the model in a turn.

6.3 Files

The Files area can browse, upload, create, download, and delete files within the allowed scope. Before deleting or replacing a file, confirm the path, Profile, project, and purpose of the resource.

6.4 Analytics

The Analytics area summarizes token usage, cache, costs, sessions, and consumption by model or period. Local counts may omit auxiliary calls, retries, and operations recorded only by the provider. These values are operational estimates and do not replace the official bill.

6.5 Models

The Models area defines the active model and tracks availability or usage. Some changes affect only new sessions; others may require restarting components. Confirm the effect in the documentation for the installed version.

6.6 Logs

The Logs area is the starting point for diagnostics. It can filter records by file, severity, component, and number of lines. Logs may contain paths, IDs, session content, and credential fragments. Raw log content should not be published.

6.7 Recurring tasks

The Cron area creates and manages recurring tasks. A task may contain a prompt, cron expression, destination, next run, pause state, and immediate-run action.

Before enabling an automation, verify Profile, permissions, tools, destination, and the impact of repeated executions.

7. Capabilities, extensions, and integrations in Dashboard

7.1 Skills

Skills group instructions, references, and specialized resources. Dashboard can search, install, create, update, enable, and disable skills depending on the version.

7.2 Plugins

Plugins extend the runtime or replace components such as the memory provider or context engine. Repository-based installations should be evaluated for origin, permissions, and executed code.

7.3 MCP

The MCP area registers local or HTTP servers, tests connections, and exposes tools and resources. Authentication, server origin, and tool set should be reviewed before activation.

7.4 Channels

The Channels area configures messaging platforms linked to the Messaging Gateway. Changes may require a gateway restart. Tokens, channel identifiers, and local paths are sensitive information.

7.5 Webhooks

Webhooks receive HTTP events and forward them to actions or channels. Because they can trigger operations without an open human conversation, they require authentication, source validation, limited scope, and observability.

7.6 Pairing

Pairing controls which users may converse with the agent on remote channels. Approval should occur before exposing tools or information to a new user.

7.7 Profiles

The Profiles area separates logical agents with their own configuration, keys, memory, sessions, skills, cron, and gateway. Confirm the active Profile before saving any change.

7.8 Configuration

The Config area provides a visual editor for config.yaml. Because the file is shared with other interfaces of the same Profile, a change made in Dashboard can affect Desktop, CLI, TUI, and associated services. Some options take effect only in new sessions or after a restart.

7.9 Secrets and authentication

The Keys area manages credentials for providers, tools, gateway, and integrations. Even masked values can reveal account existence, variable names, or authentication state. Public screenshots should hide all sensitive information.

7.10 System and maintenance

The System area centralizes diagnostics, updates, gateway, memory, credential pool, audit, backup, and checkpoints. Maintenance operations should be performed with a backup and confirmation of the affected Profile.

7.11 Interactive API

The /docs route exposes the local installation's OpenAPI specification. It is intended for integration and development. Official conceptual documentation remains the main reference for behavior and configuration.

8. CLI and TUI

8.1 CLI

The CLI receives commands typed in the shell. It is suitable for installation, configuration, diagnostics, headless servers, and reproducible automation.

Examples:

```
hermes setup
hermes model
hermes tools
hermes sessions list
hermes gateway start
hermes dashboard
hermes desktop
hermes status
```

The plain hermes command starts the classic interactive interface by default. A command with a subcommand, such as hermes status, performs a specific operation and returns the result.

8.2 TUI

The TUI is the modern text-mode interface. It uses panels, selectors, tool cards, slash commands, and keyboard navigation inside the terminal.

```
hermes --tui
```

The mode can also be set as the Profile default. When display.interface: tui or HERMES_TUI=1 is active, the plain hermes command opens the TUI. Explicit options take precedence:

```
hermes --cli
hermes --tui
```

CLI administers through commands. TUI maintains an interactive session organized visually in the terminal. Both use the same agent core.

8.3 TUI embedded in Dashboard

In Dashboard's Chat tab, the server starts the TUI behind a pseudoterminal and streams its output to the browser. Dashboard is therefore a web application, but its Chat tab contains the actual embedded TUI.

9. Messaging Gateway and remote connections

9.1 Messaging Gateway

The Messaging Gateway is a continuous process that receives and delivers messages from external platforms. It can keep Hermes available on Telegram, Discord, Slack, WhatsApp, email, and other channels even when Desktop and Dashboard are closed.

The Messaging Gateway is not:

- the model provider;
- the Web Dashboard;
- the Desktop connection screen;
- the `hermes serve` backend used by remote Desktop.

9.2 Provider, Nous Portal, and Tool Gateway

- **Model provider:** service that performs inference.
- **Nous Portal:** Nous Research account/service for access to models and related resources.
- **Tool Gateway:** service that can route hosted tools such as search, image, voice, or browser.

These components do not replace the interfaces or Messaging Gateway.

9.3 Remote backend

Desktop can connect to a hermes serve backend running on another machine. A remote service should use authentication and a protected network. Binding a service to a non-loopback address increases exposure.

On trusted networks, mechanisms such as VPN and authentication compatible with the documentation can be used. Publicly exposed services require an appropriate identity and protection strategy. Credentials should never be placed in public examples.

10. Choosing the interface

10.1 Decision matrix

Need	Recommended surface
------	---------------------

--- ---

Chat, edit a project, and follow files	Desktop
--	---------

Inspect sessions, logs, metrics, or configuration	Dashboard
---	-----------

Install, diagnose, or automate on a server	CLI
--	-----

Chat without leaving the terminal	TUI
-----------------------------------	-----

Keep the agent connected to messaging apps	Messaging Gateway
--	-------------------

Manage multiple Profiles and integrations	Dashboard
---	-----------

Run an operation in a script CLI

Follow artifacts and previews next to chat Desktop

10.2 Recommended operational flow

1. Identify the active Profile.
2. Confirm the backend and provider in use.
3. Select the interface appropriate for the task.
4. Check enabled permissions and tools.
5. Execute the task.
6. Review the result, logs, and persisted changes.
7. Record or archive the session as needed.

11. Security and best practices

11.1 Dashboard and local services

- Keep Dashboard on `127.0.0.1` when remote access is not required.
- Use authentication and a firewall when binding services to other addresses.
- Do not publish tokens, IDs, session titles, logs, or personal paths.
- Confirm the active Profile before changing Config or Keys.
- Check which changes require a new session or restart.

11.2 Tools and execution

- Use the smallest necessary tool set.
- Evaluate destructive commands before approval.
- Keep Git, backups, and checkpoints for recovery.
- Limit environment variables forwarded to tools.
- Define command time and output limits.
- Review the origin and permissions of plugins, skills, and MCP servers.

11.3 Data and observability

- Treat logs as potentially sensitive.
- Compare local metrics with the provider's bill.
- Separate credentials for models, tools, and channels.
- Archive sessions for organization; delete only after confirming scope.
- Keep persistent memory short, stable, and relevant.

12. Glossary

Agent: system combining model, instructions, context, tools, and execution rules.

Backend: process that executes and orchestrates the agent for a client interface.

CLI: command-line interface.

Context: active material sent to the model in a turn.

Dashboard: administrative panel accessed through the browser.

Desktop: native Hermes Agent application.

Gateway: generic term that must be qualified; it may mean Messaging Gateway or the Desktop connection backend depending on context.

Persistent memory: selected facts reused in future sessions.

MCP: protocol for making tools and resources available to the agent.

Model: AI system used to generate responses and decisions.

Profile: isolated logical instance with its own configuration and state.

Plugin: software extension for the runtime or interface.

Provider: service that hosts or provides access to models.

Session: persisted record of a conversation or execution.

Skill: set of specialized instructions and resources.

Tool: concrete capability used by the agent to act.

TUI: interactive text-mode interface run in the terminal.

Official references

Documentation consulted and verified on August 7, 2026:

- Hermes Desktop: <https://hermes-agent.nousresearch.com/docs/user-guide/desktop>
- Web Dashboard: <https://hermes-agent.nousresearch.com/docs/user-guide/features/web-dashboard>
- TUI: <https://hermes-agent.nousresearch.com/docs/user-guide/tui>
- CLI Commands Reference: <https://hermes-agent.nousresearch.com/docs/reference/cli-commands>
- Persistent Memory: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory/>
- Configuration: <https://hermes-agent.nousresearch.com/docs/user-guide/configuration>
- Profiles: <https://hermes-agent.nousresearch.com/docs/user-guide/profiles/>
- Official repository: <https://github.com/NousResearch/hermes-agent>

Content translated from Portuguese into English by AI.